# Recurrent networks

Last modified: 2026-10-30

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers the part of his module on recurrent networks that deals with training them, and with turning symbols into vectors. The exercises are new.

## 1 Welcome

### 1.1 Plan for this lecture

The DATA 471 notes defined recurrent layers[1], found that their gradients[2] can vanish or explode[3] over long sequences, and defined the LSTM[4] layer that keeps a memory with gates[5]. This lecture trains these layers in PyTorch:

1. the shapes that `nn.RNN` takes and returns;
2. reading a digit one row at a time, then one pixel at a time;
3. capping the size of the gradient[6];
4. LSTM[7] layers and a lighter gated layer;
5. turning symbols such as letters into vectors that a network can learn.

### 1.2 The digits as sequences

```
import math

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from torch import nn
```

[1] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-recurrent-layer
[2] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[3] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#thm-jacobian-bound
[4] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[5] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-gate
[6] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[7] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm

```
_ = torch.manual_seed(481)
digits = pd.read_csv("data/optdigits-test.csv", header=None)
pixels = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
n_train = 1200
order = torch.randperm(len(pixels))
x_train, y_train = pixels[order[:n_train]], y[order[:n_train]]
x_test, y_test = pixels[order[n_train:]], y[order[n_train:]]
loss_fn = nn.CrossEntropyLoss()
LR = 0.01
BATCH_SIZE = 32


class Reader(nn.Module):
    # Reads each image as a sequence of `step_size` pixels at a time,
    # with the recurrent layer `layer`, and classifies it from the last state.

    def __init__(self, layer, step_size, hidden_size=32):
        super().__init__()
        self.step_size = step_size
        self.recurrent = layer(step_size, hidden_size, batch_first=True)
        self.out = nn.Linear(hidden_size, 10)

    def forward(self, images):
        sequence = images.reshape(len(images), -1, self.step_size)
        states, _ = self.recurrent(sequence)
        return self.out(states[:, -1])


def make_reader(layer, step_size, seed=1):
    torch.manual_seed(seed)
    return Reader(layer, step_size)


def train(net, n_epochs=10, lr=LR, batch_size=BATCH_SIZE, seed=0, max_norm=math.inf):
    # Adam on the training digits; returns the gradient norm before each step.
    # A finite `max_norm` rescales any larger gradient to that norm.
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    generator = torch.Generator().manual_seed(seed)
    norms = []
    for epoch in range(n_epochs):
        net.train()
        shuffled = torch.randperm(n_train, generator=generator)
        for start in range(0, n_train, batch_size):
            batch = shuffled[start:start + batch_size]
            optimizer.zero_grad()
            loss_fn(net(x_train[batch]), y_train[batch]).backward()
            norm = nn.utils.clip_grad_norm_(net.parameters(), max_norm)
            norms.append(norm.item())
            optimizer.step()
    return norms


def accuracy(net):
    # Fraction of the test digits that `net` classifies correctly.
```

```
    net.eval()
    with torch.no_grad():
        return (net(x_test).argmax(dim=1) == y_test).float().mean().item()


print(make_reader(nn.RNN, 8))
```

```
Reader(
  (recurrent): RNN(8, 32, batch_first=True)
  (out): Linear(in_features=32, out_features=10, bias=True)
)
```

The exercises use the UCI optical digits [1], split as in the feedforward networks chapter, with 1,200 training images. `Reader` reads each image as a sequence, `step_size` pixels at a time, and classifies it with a linear layer on the last hidden state[8]. With `step_size=8`, each step reads one row of the image. `train()` runs Adam[9] with learning rate[10] 0.01 and batch size 32.

# 2 Recurrent layers in PyTorch

## 2.1 What a recurrent layer returns

**Exercise 2.1** (What `nn.RNN` returns)**.** `nn.RNN(8, 32, batch_first=True)` is a recurrent layer[a] with inputs of length 8 and a hidden state[b] of length 32. It returns two tensors. What are their shapes for the first 16 training digits, each read as a sequence of 8 rows?

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-recurrent-layer
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state

Solution

*Solution* 2.1.

```
recurrent = nn.RNN(8, 32, batch_first=True)
rows = x_train[:16].reshape(16, 8, 8)
states, last_state = recurrent(rows)
same_last = torch.equal(states[:, -1], last_state[0])
print(rows.shape, states.shape, last_state.shape, same_last)
```

```
torch.Size([16, 8, 8]) torch.Size([16, 8, 32]) torch.Size([1, 16, 32]) True
```

With `batch_first=True`, the input has shape $16 \times 8 \times 8$: 16 sequences of 8 steps, each step a row of 8 pixels. The first output holds the hidden state[a] after every step, with shape $16 \times 8 \times 32$. The second holds the hidden state after the last step only, with shape $1 \times 16 \times 32$. Its first dimension counts stacked layers times directions, here 1 layer read in 1 direction [2]. Its entries equal the last step of the first output. Without `batch_first=True`, PyTorch expects the steps first and the sequences second.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state

[8] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state
[9] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-adam
[10] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate

## 2.2 Row by row

**Exercise 2.2** (Reading a digit row by row)**.** Train `make_reader(nn.RNN, 8)` for 10 epochs[a] with `train()`. What fraction of the test digits does it classify correctly?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

*Solution* 2.2.

```
row_rnn = make_reader(nn.RNN, 8)
_ = train(row_rnn)
row_acc = accuracy(row_rnn)
print(row_acc)
```

```
0.9430485963821411
```

It classifies 94.3% of the 597 test digits correctly, after reading each digit in 8 steps.

## 2.3 Pixel by pixel

**Exercise 2.3** (Reading a digit pixel by pixel)**.** `make_reader(nn.RNN, 1)` reads each digit one pixel at a time, in 64 steps. Train it for 20 epochs[a] with `train()`. What fraction of the test digits does it classify correctly?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

*Solution* 2.3.

```
pixel_rnn = make_reader(nn.RNN, 1)
rnn_norms = train(pixel_rnn, n_epochs=20)
pixel_acc = accuracy(pixel_rnn)
print(pixel_acc)
```

```
0.0837520956993103
```

```
# The prose below says this reader does worse; stop the render if it does not.
assert pixel_acc < row_acc
```

It classifies only 8.4% of the test digits correctly, against 94.3% for the row reader of Solution 2.2. Both see every pixel, and the pixel reader trains for twice as many epochs[a]. What differs is the length of each sequence: 64 steps rather than 8. The DATA 471 notes bound how fast[b] a gradient[c] can shrink or grow over that many steps.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#thm-jacobian-bound
[c] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

# 3 Large gradients

## 3.1 The size of the gradient

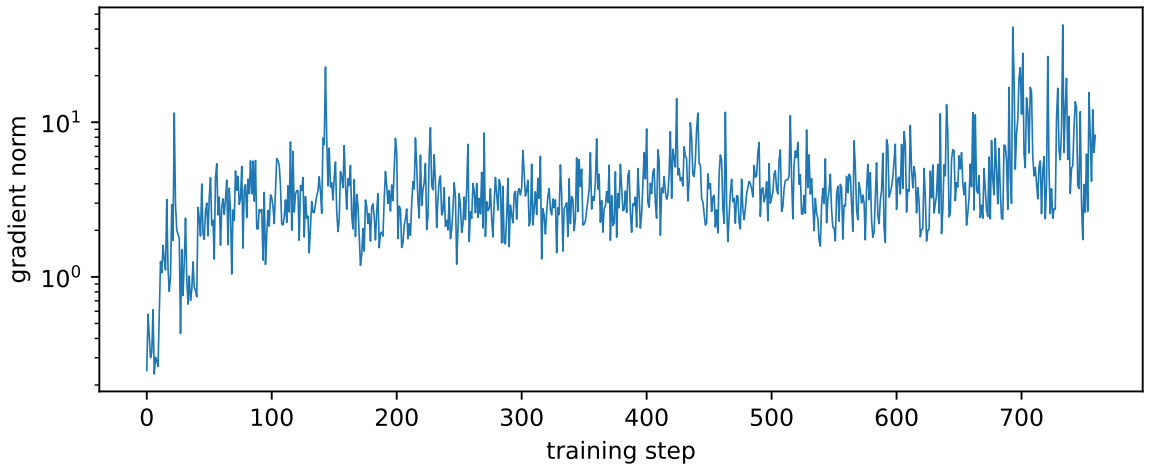
**Exercise 3.1** (The size of the gradient during training)**.** `train()` returned the norm[a] of the gradient[b] before each step of training the pixel reader in Solution 2.3, as `rnn_norms`. How large does the gradient get?

[a] https://morrison-lab.github.io/mds/linear-algebra-inner-products.html#def-norm
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

*Solution* 3.1.

```
_ = plt.figure(figsize=(7, 3))
_ = plt.semilogy(rnn_norms, linewidth=0.7)
_ = plt.xlabel("training step")
_ = plt.ylabel("gradient norm")
plt.tight_layout()
plt.show()
```




```
median_norm = np.median(rnn_norms)
max_norm_seen = max(rnn_norms)
share_above_10 = np.mean(np.array(rnn_norms) > 10)
last_steps = 100
late_median = np.median(rnn_norms[-last_steps:])
```

Figure 1: The norm[a] of the gradient[b] before each training step of the pixel reader.

The median gradient[c] norm[d] is 3.3, but the largest is 43, 13 times the median. 4% of the steps have a norm above 10. Over the last 100 steps the median norm is 5.2, and the reader ends at 8.4% test accuracy[e], near the 10% of guessing. So after the spikes the network sat in a state that training did not move it out of.

[a] https://morrison-lab.github.io/mds/linear-algebra-inner-products.html#def-norm
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[c] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

[d] https://morrison-lab.github.io/mds/linear-algebra-inner-products.html#def-norm
[e] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

## 3.2 Clipping

**Exercise 3.2** (Clipping the gradient). `train(net, max_norm=1)` applies gradient clipping[a]: before each step, `nn.utils.clip_grad_norm_` rescales any gradient[b] with norm[c] above 1 to norm 1 [3]. Trained this way for 20 epochs[d], does the pixel reader classify more test digits correctly than in Solution 2.3?

[a] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-gradient-clipping
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[c] https://morrison-lab.github.io/mds/linear-algebra-inner-products.html#def-norm
[d] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

*Solution* 3.2.

```
clipped_rnn = make_reader(nn.RNN, 1)
_ = train(clipped_rnn, n_epochs=20, max_norm=1)
clipped_acc = accuracy(clipped_rnn)
print(clipped_acc)
```

```
0.6164153814315796
```

```
# The prose below says the reader is still below the row reader.
assert clipped_acc < row_acc
```

With gradient clipping[a], the pixel reader classifies 61.6% of the test digits correctly, against 8.4% without it. So clipping helps. Clipping caps the size of each gradient[b], so no gradient Adam[c] sees can be as large as the spikes of Figure 1. The reader is still 32.7 percentage points below the row reader of Solution 2.2.

[a] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-gradient-clipping
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[c] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-adam

# 4 Gated layers

## 4.1 An LSTM

**Exercise 4.1** (An LSTM reading pixels). `make_reader(nn.LSTM, 1)` reads the pixels with an LSTM[a] layer instead, with no clipping. Trained for 20 epochs[b], does it classify more test digits correctly than the pixel reader of Solution 3.2?

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[b] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

*Solution* 4.1.

```
pixel_lstm = make_reader(nn.LSTM, 1)
lstm_norms = train(pixel_lstm, n_epochs=20)
lstm_acc = accuracy(pixel_lstm)
print(lstm_acc, max(lstm_norms))
```

```
0.8207705020904541 12.21899127960205
```

```
# The prose below says the LSTM does better; stop the render if it does not.
assert lstm_acc > max(clipped_acc, pixel_acc)
```

Yes. The LSTM[a] classifies 82.1% of the test digits correctly, against 61.6% for the clipped pixel reader. Its cell state[b] $\underset{\sim}{c}_t$ changes only by the gates'[c] products and sums, with no squashing activation between one step and the next. So a gradient[d] can pass back along the cell state through many steps when the forget gate[e] stays near 1 [4, Sec. 10.10.1]. The LSTM's largest gradient norm[f] during training was 12.2, against 43 for the pixel reader of Solution 2.3.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-gate
[d] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[e] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[f] https://morrison-lab.github.io/mds/linear-algebra-inner-products.html#def-norm

## 4.2 Two gates instead of three

**Definition 4.1** (Gated recurrent unit (GRU))**.** A **gated recurrent unit** (GRU) layer keeps only a hidden state[a] $\underset{\sim}{h}_t \in \mathbb{R}^H$, with no separate cell state[b], and two gates[c] [5]. In the form PyTorch computes [6],

$$\begin{aligned}
\underset{\sim}{z}_t &= \operatorname{expit}(\mathbf{W}_z \underset{\sim}{x}_t + \mathbf{U}_z \underset{\sim}{h}_{t-1} + \underset{\sim}{b}_z), && \text{(update gate)}\\
\underset{\sim}{r}_t &= \operatorname{expit}(\mathbf{W}_r \underset{\sim}{x}_t + \mathbf{U}_r \underset{\sim}{h}_{t-1} + \underset{\sim}{b}_r), && \text{(reset gate)}\\
\underset{\sim}{u}_t &= \tanh(\mathbf{W}_u \underset{\sim}{x}_t + \underset{\sim}{b}_u + \underset{\sim}{r}_t \odot (\mathbf{U}_u \underset{\sim}{h}_{t-1} + \underset{\sim}{c}_u)), && \text{(candidate)}\\
\underset{\sim}{h}_t &= \underset{\sim}{z}_t \odot \underset{\sim}{h}_{t-1} + (1 - \underset{\sim}{z}_t) \odot \underset{\sim}{u}_t.
\end{aligned}$$

Here $\underset{\sim}{c}_u$ is a second bias vector, and PyTorch also adds one to $\mathbf{U}_z \underset{\sim}{h}_{t-1}$ and to $\mathbf{U}_r \underset{\sim}{h}_{t-1}$. Cho et al. [5] apply the reset gate before the matrix, writing $\mathbf{U}_u(\underset{\sim}{r}_t \odot \underset{\sim}{h}_{t-1})$ in the candidate, and also put $\underset{\sim}{z}_t$ on the old state. Chung et al. [7] and the DATA 471 notes[d] put $1 - \underset{\sim}{z}_t$ on the old state instead, which swaps the meaning of $\underset{\sim}{z}_t$ near 0 and near 1.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[c] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-gate
[d] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#rem-gru

**Example 4.1** (Parameters of `nn.GRU` and `nn.LSTM`)**.**

```
def n_params(net):
    return sum(p.numel() for p in net.parameters())


gru_count = n_params(nn.GRU(1, 32))
lstm_count = n_params(nn.LSTM(1, 32))
print(gru_count, lstm_count)
```

```
3360 4480
```

A GRU layer has three blocks of weights, for $\underset{\sim}{z}_t$, $\underset{\sim}{r}_t$ and $\underset{\sim}{u}_t$, where an LSTM[a] layer has four. PyTorch gives each block two bias vectors, one added to $\widetilde{\mathbf{W}}\underset{\sim}{x}_t$ and one to $\mathbf{U}h_{t-1}$ [6]. So `nn.GRU(1, 32)` has $3 \times 32 \times (1 + 32 + 2) = 3360$ parameters[b], and `nn.LSTM(1, 32)` has $\widetilde{4} \times 32 \times (1 + 32 + 2) = 4480$.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

## 4.3 Comparing the readers

**Exercise 4.2** (A GRU reading pixels)**.** Train `make_reader(nn.GRU, 1)` for 20 epochs[a]. Does the GRU reader classify more test digits correctly than the LSTM[b] reader of Solution 4.1?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm

Solution

*Solution* 4.2.

```
pixel_gru = make_reader(nn.GRU, 1)
_ = train(pixel_gru, n_epochs=20)
gru_acc = accuracy(pixel_gru)
print(gru_acc)
```

```
0.8442211151123047
```

```
gap_digits = round(abs(gru_acc - lstm_acc) * len(y_test))
# The prose below says both gated readers beat both plain ones; stop if not.
assert min(gru_acc, lstm_acc) > max(pixel_acc, clipped_acc)
```

| Layer reading the pixels | Gradient clipping[a] | Test accuracy[a] |
|---|---|---|
| `nn.RNN` | None | 8.4% |
| `nn.RNN` | To 1 | 61.6% |
| `nn.LSTM` | None | 82.1% |
| `nn.GRU` | None | 84.4% |

The GRU reader classifies 84.4% of the test digits correctly, 2.3 percentage points above the LSTM[b] reader, with 75% as many parameters[c] in its recurrent layer[d]. So yes: the two readers differ by 14 of the 597 test digits. Both gated readers beat the plain recurrent readers. On other tasks, Chung et al. [7] found neither gated layer consistently better than the other.

[a] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-gradient-clipping
[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[b] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-lstm
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[d] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-recurrent-layer

# 5 Symbols as vectors

## 5.1 A one-hot vector times a matrix

**Exercise 5.1** (Multiplying a one-hot vector by a matrix)**.** A network that reads text needs each character as a vector. Number the 26 lowercase letters and the space from 0 to 26, so `"c"` is 2. Count vector entries and matrix rows from 0 as well. Let $\underset{\sim}{e}_2 \in \{0,1\}^{27}$ be the one-hot encoding[a] of `"c"`, and let $\mathbf{E}$ be the $27 \times 4$ matrix printed below. What is $\underset{\sim}{e}_2{}^\top \mathbf{E}$?

```
alphabet = "abcdefghijklmnopqrstuvwxyz "
_ = torch.manual_seed(2)
E = torch.randn(len(alphabet), 4).round(decimals=2)
print(E[:4])
```

```
tensor([[-1.0400,  0.9200, -1.3000, -1.1100],
        [-1.2200,  1.1700, -1.0600, -0.1200],
        [-0.9100,  0.3500, -0.5700, -0.2400],
        [ 1.0100, -0.7500, -0.2200, -0.4300]])
```

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-one-hot-encoding

Solution

*Solution* 5.1.

```
one_hot_c = nn.functional.one_hot(torch.tensor(alphabet.index("c")), len(alphabet)).float()
product = one_hot_c @ E
print(product, E[2], torch.equal(product, E[2]))
```

```
tensor([-0.9100,  0.3500, -0.5700, -0.2400]) tensor([-0.9100,  0.3500, -0.5700, -
0.2400]) True
```

Entry $j$ of $\underset{\sim}{e}_2{}^\top \mathbf{E}$ is $\sum_i (\underset{\sim}{e}_2)_i \mathbf{E}_{ij} = \mathbf{E}_{2j}$, since $(\underset{\sim}{e}_2)_i$ is 1 for $i = 2$ and 0 for every other $i$. So the product is row 2 of $\mathbf{E}$, the third row printed above. Multiplying by a one-hot encoding[a] only looks up a row. A network can skip multiplying by the 26 zeros and read the row directly.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-one-hot-encoding

**Definition 5.1** (Embedding layer)**.** An **embedding layer** maps each of $K$ symbols, such as characters or words, to a vector in $\mathbb{R}^d$. Symbol $k$ maps to row $k$ of a $K \times d$ matrix $\mathbf{E}$ of parameters[a], learned with the rest of the network [4, Sec. 12.4.2]. Row $k$ is the vector that represents symbol $k$. The layer equals a linear layer with no bias applied to the one-hot encoding[b] of the symbol, computed by looking up the

row.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-one-hot-encoding

**Example 5.1** (`nn.Embedding`).

```
embed = nn.Embedding(len(alphabet), 4)
word = torch.tensor([alphabet.index(ch) for ch in "deep"])
vectors = embed(word)
same_rows = torch.equal(vectors, embed.weight[word])
print(word, vectors.shape, same_rows)
```

```
tensor([ 3,  4,  4, 15]) torch.Size([4, 4]) True
```

`nn.Embedding(27, 4)` is an embedding layer for the 27 characters, with a $27 \times 4$ matrix `weight` [8]. It takes a tensor of whole-number indices, here the indices of the letters of `"deep"`, and returns one row of `weight` for each. For the 4 letters the result has shape $4 \times 4$. The two `"e"`s get the same vector. The returned rows equal the rows of `embed.weight` at those indices.

## 5.2 Further reading

Read

*Deep Learning*'s chapter 10[a] covers recurrent networks, LSTMs and gradient clipping [4]. Cho et al. [5] introduced the GRU, and Chung et al. [7] compared it with the LSTM.

[a] https://www.deeplearningbook.org/contents/rnn.html

# 6 References


[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[11].

[2] PyTorch Contributors, "Torch.nn.RNN." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html

[3] PyTorch Contributors, "Torch.nn.utils.clip_grad_norm_." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[5] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 conference on empirical methods in natural language processing*, 2014, pp. 1724–1734. doi: 10.3115/v1/D14-1179[12].

[6] PyTorch Contributors, "Torch.nn.GRU." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

[7] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling." 2014. Available: https://arxiv.org/abs/1412.3555

[8] PyTorch Contributors, "Torch.nn.Embedding." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.Embedding.html


[11] https://doi.org/10.24432/C50P49
[12] https://doi.org/10.3115/v1/D14-1179